

Diondria Morris

Lab 06

ITAI-3377 Spr25

AI-Driven Data Analytics for IIoT

In this lab I applied artificial intelligence methods to analyze sensor data collected in a simulated smart factory environment. The goal was to understand how machine learning and data processing techniques could be used to monitor, classify, and react to changes in its surroundings like temperature, which is crucial in industrial settings. Working in a Jupyter Notebook I built a complete data analysis and modeling pipeline. It was difficult but I got to practice data ingestion, visualization and training classification models while evaluating their performance.

The dataset contained 5,707 temperature readings, each identified by a unique ID, a room identifier, a timestamp (*noted_date*), and a binary location label indicating whether the reading was taken indoors or outdoors. To start off I imported core libraries such as *pandas*, *matplotlib*, *seaborn*, *scikit-learn* etc. I then loaded the data using *pd.read_csv()* and did a structural inspection using *.head()*, *.info()*, and *.describe()* to check for missing values and verify data types were appropriate. One essential preprocessing step was converting the *noted_date* field into *datetime* format using *pd.to_datetime()*. This allowed me to later analyze the time patterns.

When exploring the data I visualized temperature readings over time by grouping them by location. A line plot revealed a consistent trend: indoor temperatures remained relatively stable, while outdoor readings showed greater fluctuation over short intervals. This directly supported

the idea that environmental data in industrial IoT settings can vary dramatically depending on its location. The range of temperatures observed overall spanned from 16.5°C to 32.8°C but most indoor readings stayed between 22°C and 25°C, as confirmed by both the histogram and box plot. This tight clustering made sense in a controlled indoor environment, and the box plot further conveyed that outdoor readings had an interquartile range (IQR) of 6.1°C, compared to just 2.3°C indoors.

As the data size grew and visualizations became more complex, I began to encounter performance issues with my laptop. Plotting thousands of time stamped data points caused the notebook kernel to crash. I resolved this after a while by reducing the number of plotted points using `df.sample()` and narrowing the time window for clearer visuals. These moments reminded me of the need to balance visual clarity and computational cost, this could apply to systems running on resource limited platforms.

With the data prepared the next step was modeling. I selected temperature as the sole input feature (X) and used `LabelEncoder` to convert the out/in column into a binary target variable (y). I then split the data into training and testing sets using `train_test_split()` and trained three models: Logistic Regression, Decision Tree, and Random Forest. These were initialized and trained using `.fit()` and used to predict values on the test set via `.predict()`.

The results were clear:

- **Random Forest** achieved an accuracy of **96.7%**
- **Decision Tree** followed with **94.1%**
- **Logistic Regression** trailed at **89.5%**

The accuracy bar chart made these differences easy to interpret. Random Forest's superior performance supported the idea that ensemble models handle variability better, even when using a single numeric feature. I also used a confusion matrix to confirm that the Random Forest model had fewer false positives and negatives than the others. This information made me think about why certain models are better suited for specific data conditions.

To deepen my interpretation, I linked the statistical trends from my visualizations with model behavior. For instance, the outdoor label's greater variability likely introduced more uncertainty in the classifier's boundary. This is aligned with Farag et al. (2023), who discusses how data heterogeneity in IIoT networks can impact AI reliability and Age of Information (AoI) metrics. In the industrial world having a model that can reliably classify noisy data, like outdoor readings, is valuable for instant action.

In conclusion, this lab was an exercise in full stack AI development. I worked through the challenges of managing detection data, building models, and interpreting statistical patterns in a meaningful way. Each step from label encoding and datetime conversion to box plots and accuracy evaluation, forced me to dive into how AI operates in smart environments. Despite platform issues and model tuning challenges I was able to draw strong connections between my code, my results, and the broader role of AI in industry. This experience strengthened my coding and analysis skills and also my confidence in interpreting AI results within complex systems.